

- CareerOrbit V3.1 — Technical Test Cases & Validation Bible
 - 1. The Validation Philosophy
 - 2. The 9-Step Core Test Suite
 - **1** Database & Schema (01_db_schema.test.ts)
 - **2** Server Actions & Security (02_auth_actions.test.ts)
 - **3** Onboarding & UI Flow (03_onboarding_ui.test.tsx)
 - **4** AI Generation Logic (04_ai_generation.test.ts)
 - **5** Adaptive Brain Math (06_core_loop.test.ts)
 - **6** High-Fidelity BKT Engine (09_bkt_engine.test.ts)
 - 3. BKT Mathematical Invariants (The "Golden Cases")
 - Case A: The "Lucky Guesser" (P-G Barrier)
 - Case B: The "Cognitive Breakthrough" (Recovery Boost)
 - Case C: The "Slip Protection" (P-S Buffer)
 - Case D: Asymptotic Resilience
 - 4. LLM Response Quality Benchmarking
 - 5. End-to-End QA Checklist

CareerOrbit V3.1 — Technical Test Cases & Validation Bible

Audience: QA Engineers, AI Researchers, Backend Developers **Objective:** A comprehensive reference for the 9-step validation suite and the mathematical invariants governing the Mastery Engine.

1. The Validation Philosophy

CareerOrbit is not just a UI; it is a **scientifically valid educational instrument**. We test for two types of integrity:

1. **Architectural Integrity:** Does the data flow correctly between the AI, DB, and UI?
2. **Pedagogical Integrity:** Is the BKT engine correctly identifying mastery and ZPD?

2. The 9-Step Core Test Suite

Located in `testing_scripts/`. Run via `npx vitest run`.

1 Database & Schema (`01_db_schema.test.ts`)

- **Case:** Connection & Drizzle Alignment.
- **Verification:** Ensures the TypeScript models perfectly match the Neon PostgreSQL tables.

2 Server Actions & Security (`02_auth_actions.test.ts`)

- **Case:** Null Injection Prevention.
- **Verification:** Prevents database "null pollution" by catching partial form submissions.

3 Onboarding & UI Flow (`03_onboarding_ui.test.tsx`)

- **Case:** Question Carousel Progress.
- **Verification:** Confirms the `1 / 8` progress indicator and button enabling logic.

4 AI Generation Logic (`04_ai_generation.test.ts`)

- **Case:** Schema Enforcement.
- **Verification:** Mocks the AI SDK to ensure generated roadmaps match the `Zod moduleSchema` exactly.

5 Adaptive Brain Math (`06_core_loop.test.ts`)

- **Formula Validation:** `scoreDelta = (score - 50) * 0.4`.

- **Attempt Penalty:** Verifies a -5 point deduction for second attempts.
- **Trigger Logic:** Confirms the 3 behavioral triggers:
 - **Stuck:** Time > 900s.
 - **Repeated Failure:** Fail Count >= 2.
 - **Performing Well:** Pass Count >= 3.

6 High-Fidelity BKT Engine (09_bkt_engine.test.ts)

This is the most critical test file. It validates the Hidden Markov Model logic.

3. BKT Mathematical Invariants (The "Golden Cases")

Case A: The "Lucky Guesser" (P-G Barrier)

- **Scenario:** User knows nothing ($pM = 0.10$), gets a 'Hard' question right.
- **Invariant:** Mastery **cannot** jump above 0.50 on a single success. The pG (Guess) penalty must correctly discount the success.

Case B: The "Cognitive Breakthrough" (Recovery Boost)

- **Scenario:** User is at $pM < 0.20$ and provides a correct answer.
- **Invariant:** The system must apply the **Recovery Boost** coefficient (halving the pG penalty), leading to a higher mastery jump than standard BKT.

Case C: The "Slip Protection" (P-S Buffer)

- **Scenario:** Expert user ($pM = 0.90$) misses an 'Easy' question.
- **Invariant:** Mastery **cannot** drop below 0.50 on a single failure. The pS (Slip) parameter protects the user's progress from a simple typo.

Case D: Asymptotic Resilience

- **Invariant:** Mastery is strictly clamped to $[0.001, 0.999]$. This ensures the Markov chain never locks, allowing a user to recover even from a string of 100 failures.
-

4. LLM Response Quality Benchmarking

We utilize `LLM-testing/test_results.xlsx` to track model performance over time.

- **Metric 1: Schema Pass Rate.** Percentage of responses that match the Zod schema on the first attempt.
 - **Metric 2: NOS Validity.** Verifies if generated NOS codes follow the `XXX/NXXXX` format.
 - **Metric 3: Language Consistency.** Checks if 100% of the response is in the user's target language.
-

5. End-to-End QA Checklist

For every major release, a developer must execute the following manual loop:

1. **The "Fresh Start" Loop:** Create a new user -> Complete Onboarding -> Select Path.
2. **The "Struggle" Loop:** Intentionally fail 2 quizzes -> Verify AI Mentor auto-opens with "Repeated Failure" greeting.
3. **The "Pivot" Loop:** Switch career paths mid-way -> Verify old roadmap is archived correctly.
4. **The "Mastery" Loop:** Master 3 subtopics -> Verify Radar Chart in Dashboard reflects growth.